



ARA0110 PADRÕES DE PROJETO

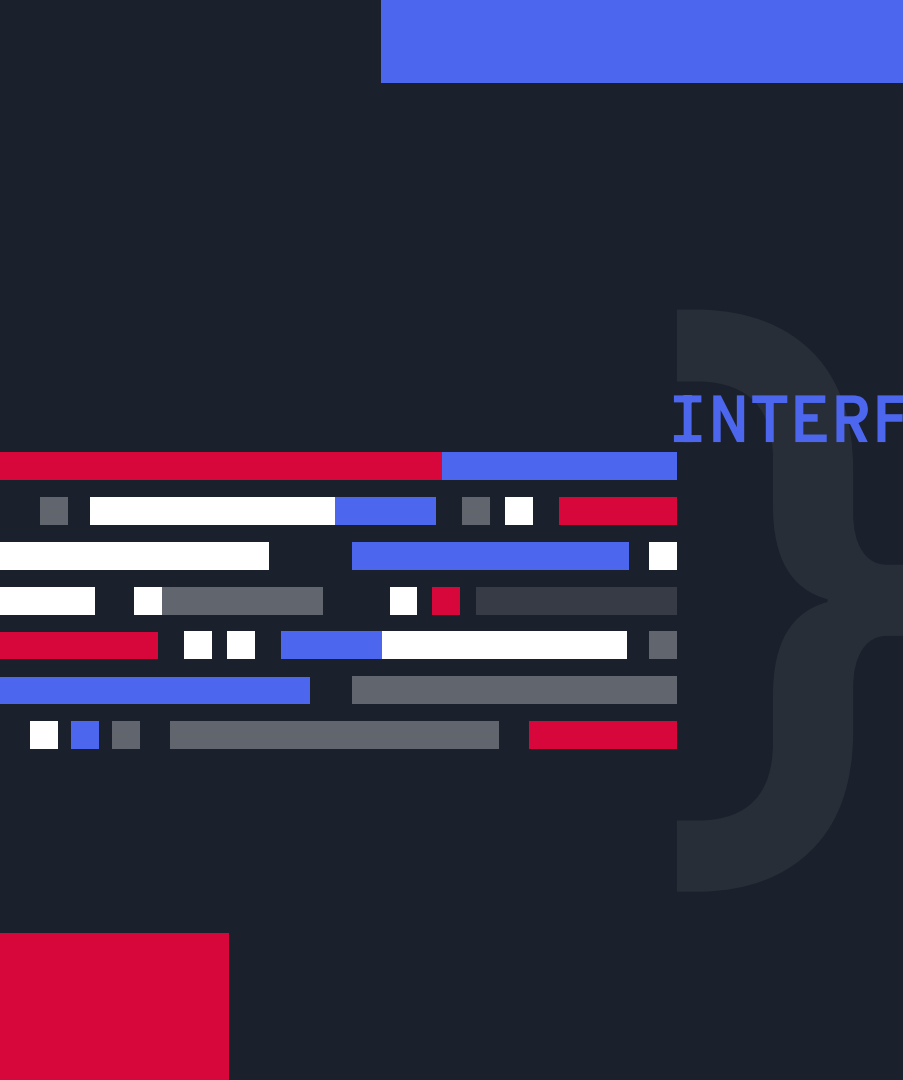
Aula 02: Padrões de projeto criacionais - parte 1





RECAPITULANDO





INTERFACES X IMPLEMENTAÇÕES

O principal desafio quando se está projetando um software orientado a objetos é a ideia de se programar para uma interface ao invés de uma implementação.

A ideia é fazer uso massivo de abstrações para modelar os componentes do software.

INTERFACES X IMPLEMENTAÇÕES

```
public class DbConnection {  
  
    public String execute() {  
        return "SELECT * FROM users";  
    }  
}
```

```
public class Repository {  
  
    public final DbConnection conn;  
  
    public Repository(DbConnection conn) {  
        this.conn = conn;  
    }  
  
    public String findUsers() {  
        return this.conn.execute();  
    }  
}
```

```
public class Main {  
  
    public static void main(String[] args) {  
        DbConnection conn = new DbConnection();  
        Repository repo = new Repository(conn);  
  
        System.out.println(repo.findUsers());  
    }  
}
```



INTERFACES X IMPLEMENTAÇÕES

```
public interface DbConnection {  
    String execute();  
}  
  
public class MySQLDbConnection implements DbConnection {  
  
    @Override  
    public String execute() {  
        return "SELECT * FROM users";  
    }  
}
```

```
public class Repository {  
    public final DbConnection conn;  
    public Repository(DbConnection conn) {  
        this.conn = conn;  
    }  
  
    public String findUsers() {  
        return this.conn.execute();  
    }  
}
```

```
public class Main {  
  
    public static void main(String[] args) {  
        DbConnection conn = new MySQLDbConnection();  
        Repository repo = new Repository(conn);  
  
        System.out.println(repo.findUsers());  
    }  
}
```

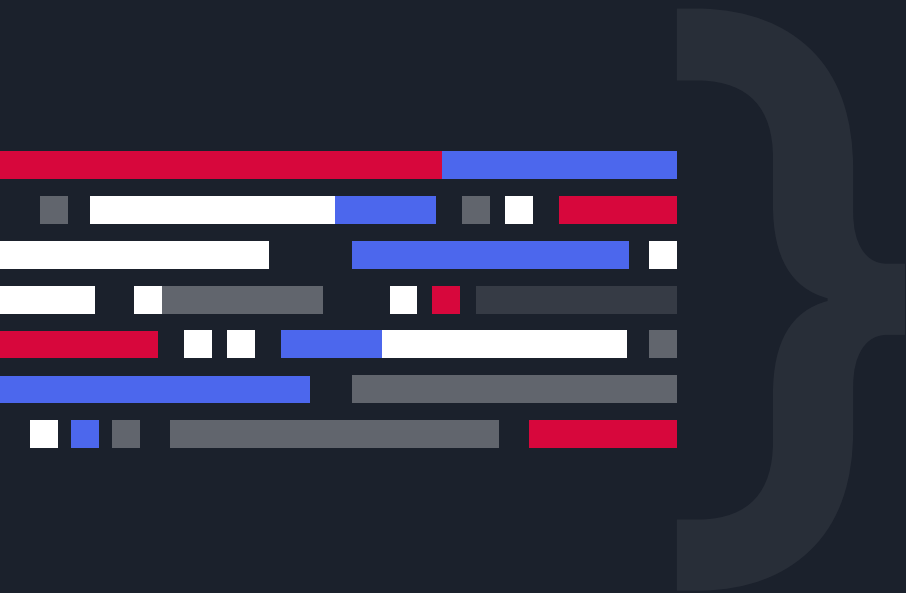




PADRÕES DE PROJETOS

O grupo GoF definiu em seu livro uma classificação para agrupar padrões de projeto de acordo com sua finalidade. São elas:

- Padrões criacionais
- Padrões estruturais
- Padrões comportamentais



PADRÕES CRIACIONAIS

Os padrões de projeto criacionais focam na instanciação de objetos, tornando o sistema independente de criação, representação e composição dos objetos.

Nesses padrões, encontramos diversas abordagens de criação de objetos.

ABSTRACT FACTORY





DEFINIÇÃO

Fornecer uma interface para criação de famílias de objetos relacionados ou dependentes sem especificar suas classes concretas.

GoF, Padrões de Projetos. p.95

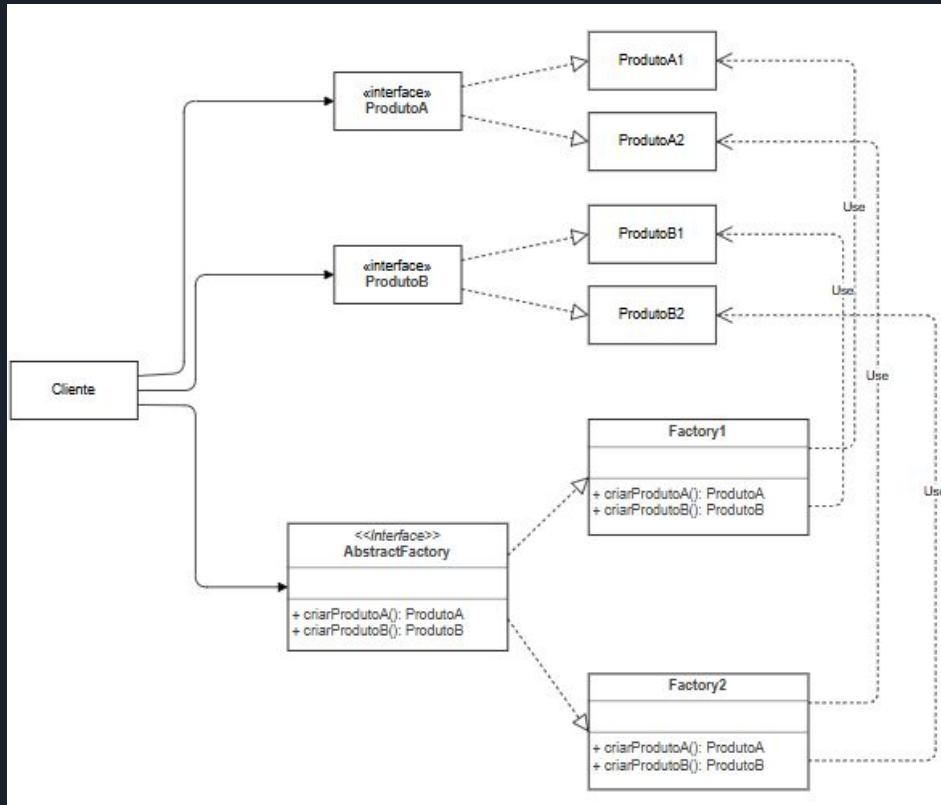


MOTIVAÇÃO

Considere um cenário em que estamos implementando um toolkit de interface de usuários com suporte a múltiplos estilos de interação.

Com diferentes estilos de interação, temos o desafio de trabalhar com diferentes apresentações e comportamentos para widgets da interface.

DIAGRAMA





PARTICIPANTES

- **AbstractFactory:** interface que declara as rotinas de criação de objetos-produto
- **Product:** interface que declara um objeto-produto abstrato
- **ProductX:** implementação de um objeto produto que pertence a uma determinada família de objetos-produto



PARTICIPANTES

- **FactoryX:** implementação de uma fábrica de objetos-produto de uma determinada família
- **Cliente:** elemento que faz uso apenas das interfaces que declaram os objetos-produto e a fábrica de objetos



UTILIZAÇÃO

Normalmente uma única instância de fábrica concreta é criada em tempo de execução, sendo responsável por criar objetos de uma família específica.

Nesse cenário, a criação de objetos-produto é adiada para as subclasses de fábrica concreta.



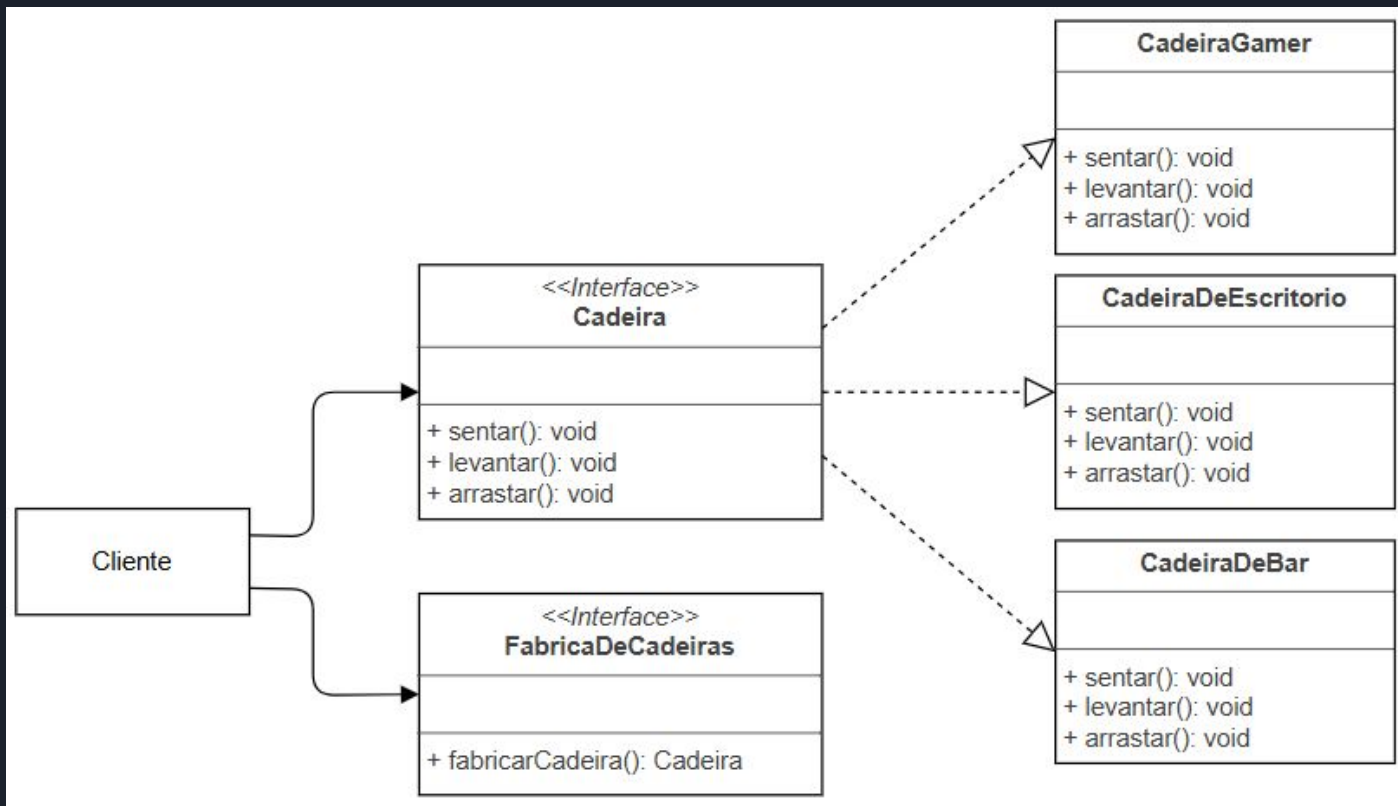
HORA DE EXERCÍCIO

Imagine uma fábrica de cadeiras que produz diferentes estilos de cadeiras para o mercado (gamer, escritório, barzinho etc).

Temos diferentes estilos de cadeiras para fabricar, onde o resultado é um tipo específico de cadeira para o mercado.

Vamos implementar esta fábrica.

DIAGRAMA





HORA DE EXERCÍCIO

Quando estamos desenvolvendo software seguindo especificações de mercado, uma das atividades mais comuns é a conexão de uma aplicação com um banco de dados.

Por se tratar de especificações de mercado, é interessante que as bibliotecas de conexão dêem suporte a diversas implementações de bancos de dados, mantendo uma flexibilidade na utilização dentro das aplicações.

DIAGRAMA?



APLICABILIDADE

- Um sistema deve ser independente de como seus produtos são criados
- Um sistema deve ser configurado com uma família de múltiplos produtos
- Uma família de objetos foi projetada para ser usada em conjunto
- Fornecer uma biblioteca de classes de produtos



CONSEQUÊNCIAS

- Isolamento das classes concretas
- Facilidade na troca de família de produtos
- Promove harmonia entre produtos
- É difícil de suportar novos tipos de produtos



BUILDER





DEFINIÇÃO

Separar a construção de um objeto complexo da sua representação, de modo que o mesmo processo de construção possa criar diferentes representações.

GoF, Padrões de Projetos. p.104

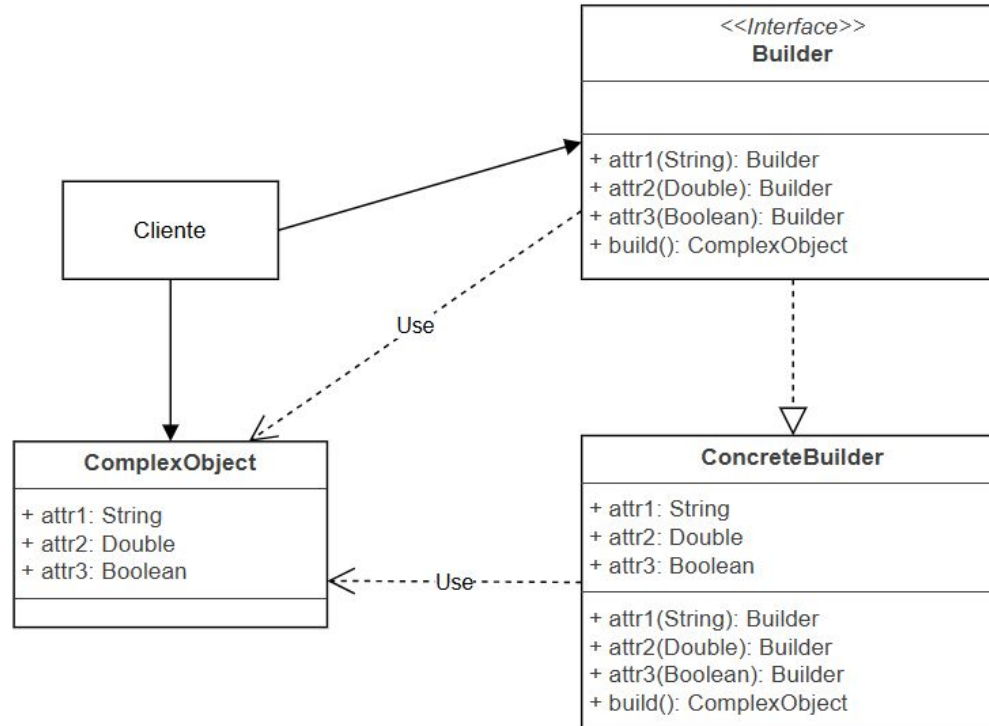


MOTIVAÇÃO

Considere a criação de um objeto que representa dados de um produto movimentado pelo setor de logística. Dependendo do produto, temos uma quantidade considerável de parâmetros para configurar na criação dessa representação, o que pode causar confusões.

Esse é um ótimo cenário para aplicação do padrão Builder.

DIAGRAMA





PARTICIPANTES

- **ComplexObject:** objeto que possui um processo de criação complexo
- **Builder:** interface que declara um construtor de objetos complexos
- **ConcreteBuilder:** implementação de um construtor de objetos complexos
- **Cliente:** elemento que faz uso da abstração de construção de objetos



UTILIZAÇÃO

O cliente utiliza uma instância de construtor de objetos e, ao longo do fluxo, ocorrem interações para preenchimento de partes do objeto de forma contínua.

Ao final, o cliente solicita ao construtor a criação do objeto final devidamente preenchido, recuperando-o.

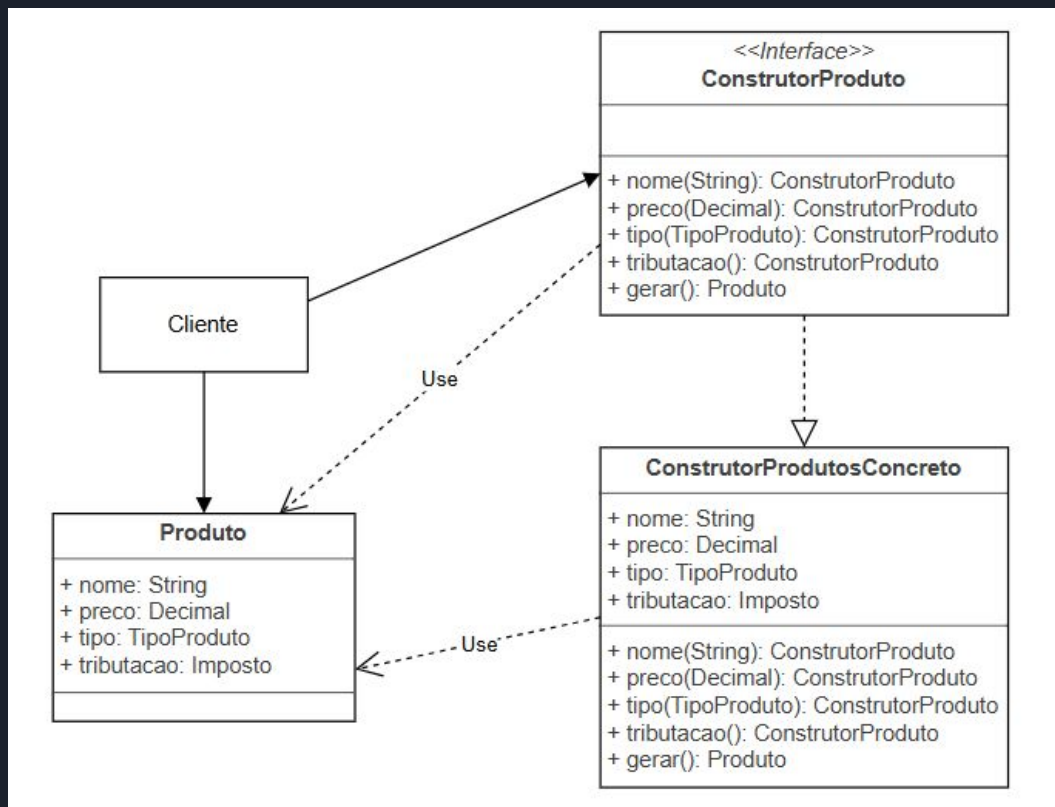


HORA DE EXERCÍCIO

Voltando ao problema da movimentação de produtos no setor de logística, vamos implementar uma interação de criação de objetos de produtos configurando os parâmetros de acordo com a necessidade.

Vamos utilizar o diagrama a seguir para basear nossa implementação.

DIAGRAMA





HORA DE EXERCÍCIO

Vamos implementar novamente o problema anterior, agora seguindo uma outra abordagem de criação de objetos.



HORA DE EXERCÍCIO

Em uma arquitetura orientada a domínio, nós temos a atividade comum de converter objetos da camada de aplicação para a camada de domínio.

As abstrações das camadas de aplicação e domínio possuem comportamentos ligeiramente diferentes, com uma certa complexidade envolvida entre elas.

DIAGRAMA?



APLICABILIDADE

- Um sistema que faz uso de objetos complexos para criação
- Um sistema que faz uso extensivo de objetos muito carregados
- Um sistema que envolve lógicas relevantes para criação de objetos



CONSEQUÊNCIAS

- Permite variar a representação interna de um objeto
- Isolar o código de construção e representação
- Oferece um controle mais fino do processo de construção
- Declaração de uma interface às vezes desnecessária para o contexto

FACTORY METHOD





DEFINIÇÃO

Definir uma interface para criar um objeto, mas deixando as subclasses decidirem que classe instanciar. O Factory Method permite adiar a instanciação para subclasses.

GoF, Padrões de Projetos. p.112

PERAE

FACTORY METHOD?



ABSTRACT FACTORY?

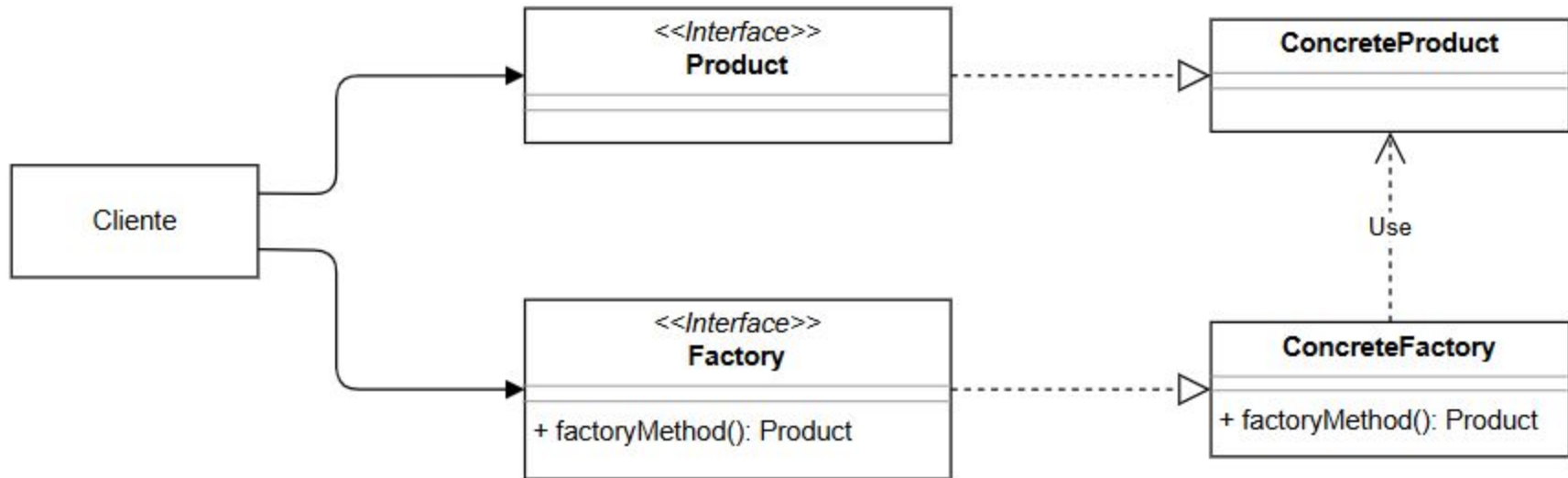


MOTIVAÇÃO

Imagine uma operação de extração de múltiplos documentos para um usuário. Quando este usuário solicita a exportação dos dados de um relatório, podem ser apresentados diversos formatos de arquivo para emitir o relatório.

Mediante a escolha do usuário, a aplicação prepara a formatação do arquivo com os exatos dados do relatório.

DIAGRAMA





PARTICIPANTES

- **Factory:** interface que declara a operação de fabricação de um objeto
- **ConcreteFactory:** implementação da operação de fabricação de objetos
- **Product:** interface que abstrai a representação de um objeto
- **ConcreteProduct:** implementação da classe de representação de um objeto
- **Cliente:** elemento que faz uso da abstração de fabricação de objetos



UTILIZAÇÃO

O cliente utiliza uma instância de fabricação de objetos de acordo com uma determinada operação, sem saber de forma direta qual tipo de objeto está sendo criado.

O que ele sabe é que o objeto retornado estará de acordo com a necessidade do fluxo de execução.

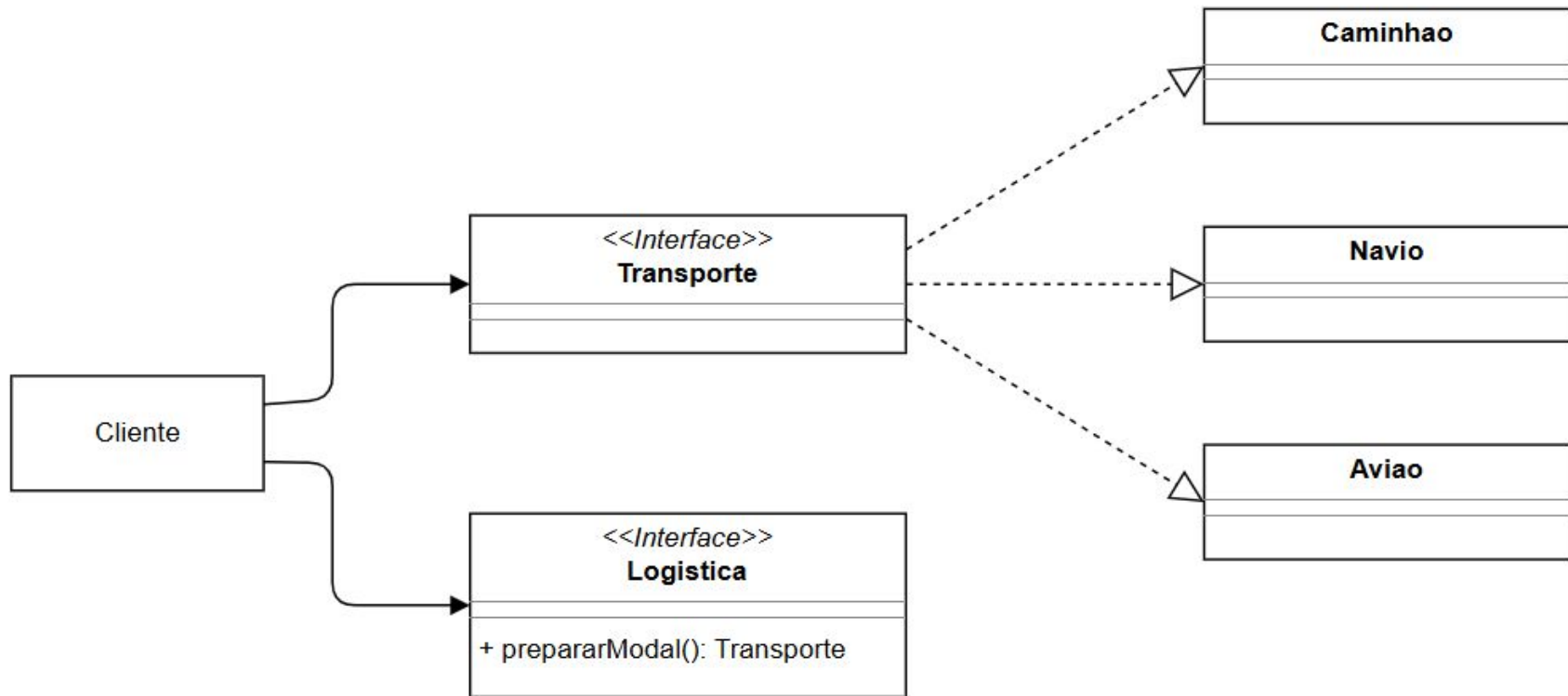


HORA DE EXERCÍCIO

Uma empresa de logística vai começar a realizar entregas de mercadorias em diferentes modais, de acordo com o tipo de frete praticado.

Por exemplo, a empresa começa realizando entregas de caminhão, mas depois passa a fazer uso de entregas com navio e avião. O sistema de gestão de frete deve ser capaz de gerar o modal de transporte de forma transparente, de acordo com o tipo de frete.

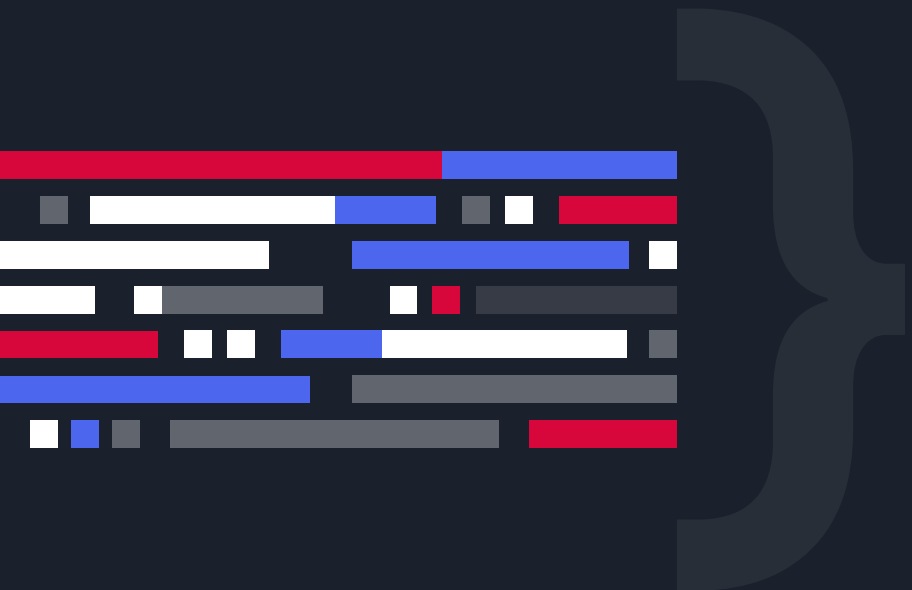
DIAGRAMA





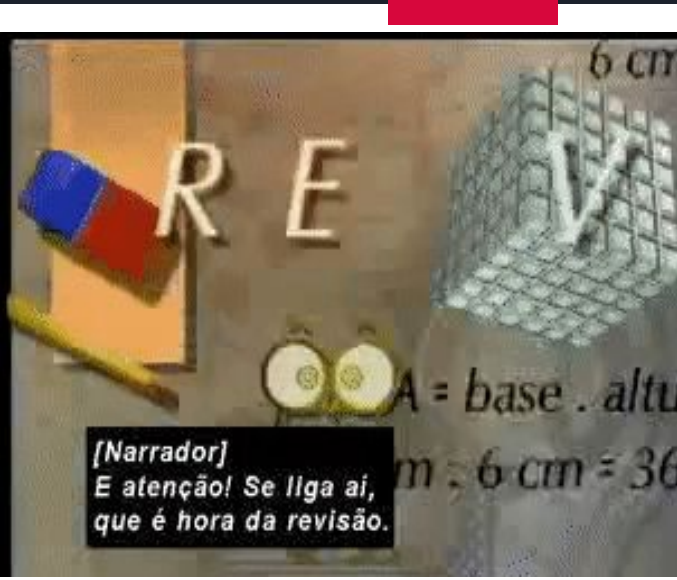
APLICABILIDADE

- Uma classe não pode antecipar a classe de objetos que deve criar
- Uma classe quer que suas subclasses especifiquem os objetos que deve criar
- Classes delegam responsabilidade para uma dentre várias subclasses auxiliares, e você quer localizar o conhecimento de qual subclasse auxiliar que é delegada



CONSEQUÊNCIAS

- Fornecer ganchos para subclasses
- Conecta hierarquia de classes paralelas
- Oferece um controle mais fino do processo de construção



RECAPITULANDO

- Programar para uma interface ao invés de uma implementação
- Padrões de projeto criacionais
- Padrão Abstract Factory
- Padrão Builder
- Padrão Factory Method

OBRIGADO!



"NÃO EXISTE TRABALHO RUIM, O RUIM É TER QUE TRABALHAR"
Seu Madruga